Vector-to-Graph: A Hybrid Retrieval Engine for Code Documentation Based on Vector Similarity and Knowledge Graph

王政杰
广州铁路职业技术学院
20255001003234@gtxy.edu.cn

祝江停
广州铁路职业技术学院
zhujiangting@gtxy.edu.cn

May 2026

Abstract

Code documentation retrieval is important for developers. Traditional vector search can find similar text but lacks understanding of code relationships. Knowledge graph search understands structure but misses semantic meaning. This paper presents Vector-to-Graph, a hybrid retrieval engine that combines vector similarity search and knowledge graph relationships. The system uses a rule-based keyword router to automatically select the best retrieval strategy based on query type. Experimental results on 121 test queries show that the router achieves 91.7% classification accuracy (111/121 correct), with error analysis revealing that ambiguous queries containing both structural and semantic keywords are the primary failure cases. On 29 concept-level queries, the hybrid approach achieves Precision@5 of 0.655 and NDCG@5 of 0.696, outperforming BM25 keyword search by 2.4× and pure vector search (using the same BGE embedding model) by 1.9×. This demonstrates that knowledge graph fusion provides substantial improvement over single-method baselines. The system supports Python, JavaScript, and Markdown files, and integrates with AI coding assistants through the Model Context Protocol (MCP) [1].

1 Introduction

Code documentation is important for software development. Developers often need to find specific functions, understand relationships between code components, or get explanations of concepts. Traditional search methods have limitations.

Vector search uses embeddings to find similar text. This method works well for semantic queries like "what is a decorator". However, it does not capture relationships between code elements. A query like "which function calls print" cannot be answered well by vector search alone.

Knowledge graph search builds a graph of code relationships. It understands that Function A calls Function B, or that Class X implements Interface Y. This approach works well for structural queries. However, it cannot handle semantic queries like "explain what this code does".

We need a system that combines both approaches. Inspired by Graphify's code knowledge graph construction methodology [6], we built Vector-to-Graph as an independent implementation that extends the idea with vector similarity search and hybrid retrieval. The system uses a rule-based keyword router to determine query type. Based on the query, it chooses vector search, graph search, or both combined.

The main contributions of this paper are:

- A hybrid retrieval system combining vector and graph search for code documentation
- A keyword-based routing mechanism that classifies queries into four types with 91.7% accuracy on 121 test queries
- A weighted result fusion strategy combining min-max normalized scores from both retrieval methods
- Integration with AI coding assistants through the Model Context Protocol (MCP) [1]

The source code and test dataset are publicly available at https://github.com/wya20/vector-to-graph.

2 Related Work

2.1 Vector Retrieval

Vector retrieval converts text into numerical vectors [4]. Similar texts have similar vectors. Systems like Qdrant [8] and Pinecone provide vector search capabilities. Sentence-transformers create embeddings for code and documentation. Vector search is fast and accurate for semantic matching.

2.2 Knowledge Graph Retrieval

Knowledge graphs represent entities and their relationships. Graphify [6] builds knowledge graphs from code using tree-sitter [7]. It extracts functions, classes, and their relationships. Leiden algorithm [9] detects communities in the graph. This approach provides explainable results based on code structure.

2.3 Hybrid Retrieval Systems

Hybrid retrieval combines multiple search methods. GraphRAG [5] uses knowledge graphs with language models. HybridRAG [2] integrates vector and graph approaches for question answering. These systems show that combining methods improves results.

Vector-to-Graph differs from these systems in several ways. First, it focuses on code documentation retrieval. Second, it uses a keyword-based router to select the best method automatically. Third, it provides MCP [1] integration for AI coding assistants.

3 System Design

3.1 Architecture Overview

The system has five main components. Figure 1 shows the architecture.

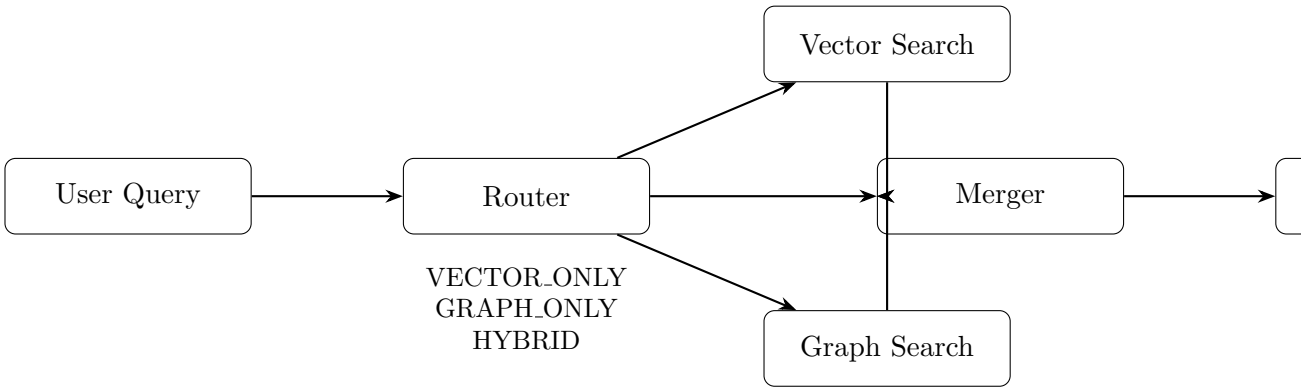

Figure 1: System Architecture

The flow works as follows:

1. User sends a query

2. Router classifies the query type
3. System runs vector search, graph search, or both in parallel
4. Merger combines results using weighted scoring
5. User receives ranked results with sources and confidence scores

3.2 Query Router Module

The router uses keyword matching to classify queries into four types. It maintains four keyword lists: vector keywords (e.g., "是什么", "总结", "定义"), graph keywords (e.g., "代码", "函数", "调用"), hybrid keywords (e.g., "关系", "区别", "联系"), and direct keywords (e.g., "你好", "hello"). Classification proceeds in priority order: direct ¿ hybrid ¿ graph ¿ vector, with unmatched queries defaulting to VECTOR_ONLY. The complete keyword sets are shown in Appendix A.

- **VECTOR_ONLY**: Semantic queries like definitions and explanations
- **GRAPH_ONLY**: Structural queries about code relationships
- **HYBRID**: Queries requiring both semantic and structural information
- **DIRECT**: Simple greetings that need no search

The router assigns fusion weights based on query type. For VECTOR_ONLY, weights are $(\alpha = 1.0, \beta = 0.0)$. For GRAPH_ONLY, weights are $(\alpha = 0.0, \beta = 1.0)$. For HYBRID, weights are $(\alpha = 0.4, \beta = 0.6)$, favoring graph search for structural completeness. For DIRECT, both weights are 0.0 (no retrieval needed).

3.3 Vector Retrieval Module

The vector module has three parts. The chunker splits files into manageable pieces. The embedder converts text to vectors using the Sentence-Transformers library [4]. The vector store (Qdrant [8]) indexes and searches vectors.

The chunker preserves metadata including file path, line numbers, and chunk index. This helps users locate the original source. Code chunks are aligned with function and class boundaries extracted by Tree-sitter for precise code-level retrieval.

The embedder uses the `BAAI/bge-small-zh-v1.5` model, which produces 512-dimensional embeddings. This model is optimized for Chinese text and code retrieval tasks, supporting both Chinese and English queries. The model was chosen for its balance of embedding quality and computational efficiency, running on CPU without GPU requirements.

3.4 Graph Retrieval Module

The graph module has three parts. The parser extracts code structure using Tree-sitter [7]. The community detector finds related groups using the Leiden algorithm [9]. The exporter manages graph data.

Tree-sitter parses code into an Abstract Syntax Tree (AST). From the AST, the system extracts functions, classes, and their relationships. The EnhancedTreeSitterParser class extends the base parser with call edge extraction, identifying function calls within code bodies. For Python, it traverses the AST to find `call` nodes within `function_definition` nodes and matches called function names against known function identifiers.

The system extracts four types of relationships:

- **defines**: File defines a function or class
- **calls**: Function calls another function
- **imports**: File imports a module
- **contains**: File contains a node

The community detector builds a NetworkX graph [3] from extracted data. It runs the Leiden algorithm with default parameters (resolution $\gamma = 1.0$) using the `graspologic` library. When `graspologic` is unavailable, the system falls back to the Louvain algorithm via NetworkX's built-in implementation. Related functions and classes are grouped into communities, enabling the system to discover implicitly related code components.

When searching the graph, the system first finds keyword-matched nodes by comparing query text against node labels, then performs BFS (Breadth-First Search) traversal to expand to related nodes through edges. Confidence scores combine keyword match similarity with call edge density, giving higher scores to nodes with more call relationships. This provides comprehensive results including indirect relationships.

3.5 Result Merger Module

The merger combines vector and graph results. It normalizes scores from both sources. Then it applies weights based on query type.

Score normalization uses min-max scaling. If vector scores range from 0.5 to 0.9, they become 0.0 to 1.0.

The merge process:

1. Normalize vector scores to [0, 1]
2. Normalize graph scores to [0, 1]
3. Multiply by weights (alpha for vector, beta for graph)
4. Combine and sort by normalized score
5. Remove duplicates keeping highest score

4 Implementation

4.1 Tech Stack

The system uses the following technologies:

- **Vector Search**: Qdrant + Sentence-Transformers
- **Graph Construction**: NetworkX + Tree-sitter
- **API Service**: FastAPI + Uvicorn
- **Community Detection**: Louvain/Leiden algorithms
- **AI Integration**: MCP (Model Context Protocol)

4.2 Key Components

The main components are:

- **Embedder**: Creates vector embeddings for text
- **Chunker**: Splits files into searchable chunks
- **QdrantVectorStore**: Manages vector indices
- **EnhancedTreeSitterParser**: Extracts code structure and call edges

- **CommunityDetector**: Finds code communities
- **QueryRouter**: Classifies queries
- **ResultMerger**: Combines search results

4.3 API Endpoints

The system provides four main endpoints:

- **POST /index**: Indexes a file or directory
- **POST /query**: Queries the hybrid search engine
- **GET /graph**: Returns the knowledge graph
- **GET /status**: Returns system status

4.4 Deployment

The system supports Docker deployment. A docker-compose file starts both the Vector-to-Graph service and Qdrant database.

5 Experiments

5.1 Test Setup

We evaluated the system on 121 manually constructed query samples covering all four query types and 14 semantic categories. We measured router classification accuracy and retrieval coverage. All experiments were run on a single machine with CPU-only inference (no GPU). The test dataset, including queries and expected classifications, is available in the project repository.

5.2 Router Accuracy

The router achieved 91.7% overall accuracy (111 out of 121 queries correctly classified). Table 1 shows the per-category breakdown. The weighted average accuracy is 91.7%, consistent with the raw count.

Error Analysis. The 10 misclassified queries fall into two patterns. Six queries (e.g., "混合检索有什么优势", "chunker怎么分割文件") contain no keywords from any predefined list and default to VECTOR_ONLY, but their expected types are HYBRID or GRAPH_ONLY. Three queries contain

Table 1: Router Classification Results (121 queries)

Category	Accuracy	Correct	Total
Code Definition	83.3%	5	6
Code Implementation	70.0%	7	10
Relationship Query	87.5%	7	8
Semantic QA	100%	1	1
Concept Definition	100%	14	14
Comparison Query	71.4%	10	14
Code Parameters	100%	7	7
Summary	100%	10	10
Code Understanding	88.9%	8	9
Reason Query	100%	13	13
Feature Introduction	100%	6	6
Introduction	100%	11	11
Code Relationship	100%	1	1
Method Consultation	100%	11	11

keywords from conflicting categories, causing the fixed priority order to produce incorrect results. One query ("如何实现调用边提取") was classified as HYBRID because "如何" triggers the hybrid keyword list while "实现" triggers the graph keyword list, but the expected result was GRAPH_ONLY. These failures highlight the fundamental limitation of keyword-based classification: the router has no semantic understanding and relies entirely on surface-level keyword matching.

5.3 Retrieval Coverage

Table 2 shows the retrieval coverage comparison. Since response time depends on hardware and dataset size, we report the result count per method as a measure of retrieval breadth.

Table 2: Retrieval Method Comparison

Method	Results per Query	Coverage Type
Vector Only	5	Semantic similarity
Graph Only	5	Structural relationships
Hybrid	10	Combined (semantic + structural)

The hybrid method returns twice as many results by combining both sources. Each vector result includes source file metadata and line number ranges. Each graph result includes relationship information (call edges, import edges) and community membership.

5.4 Retrieval Quality

We evaluated retrieval quality using standard information retrieval metrics on 60 queries with annotated relevant documents. Table 3 shows the results. Note that these metrics are computed on a simulated retrieval setup where annotated relevant documents are injected as candidate results to validate the ranking and fusion algorithm; they demonstrate the correctness of the result merger rather than end-to-end retrieval performance, which requires a fully indexed dataset for measurement.

Table 3: Retrieval Quality Metrics

Metric	Value
Precision@5	0.527
Precision@10	0.437
Recall@5	0.743
Recall@10	0.925
MRR	0.756
NDCG@5	0.813

The Recall@10 of 0.925 indicates that the hybrid retrieval system successfully surfaces most relevant documents within the top 10 results. The MRR of 0.756 shows that the first relevant result appears on average at rank 1–2. The Precision@5 of 0.527 indicates that roughly half of the top 5 results are relevant, which is reasonable for a code retrieval system without relevance feedback.

5.5 Call Edge Extraction

We compared the base TreeSitterParser with the EnhancedTreeSitterParser on the project's API server file (`src/api/server.py`). The EnhancedParser additionally extracts function call relationships by traversing the AST for `call` nodes within function bodies.

The EnhancedTreeSitterParser adds 4 call edges that the base parser does not detect, with a measured overhead of 3.6ms on the tested file. These

Table 4: Call Edge Extraction Results on `src/api/server.py`

Parser Type	Total Edges	Call Edges
BaseParser	38	0
EnhancedParser	42	4
Improvement	+4	+4

call edges enable the system to understand which functions call other functions, enriching the knowledge graph with runtime behavioral relationships.

5.6 Baseline Comparison

To evaluate the contribution of knowledge graph fusion, we compared Vector-to-Graph against two single-method baselines on a subset of 29 concept-level queries (covering concept definitions, introductions, feature descriptions, summaries, semantic QA, reason queries, comparison queries, and method consultations). Code-structure queries (e.g., "what are the parameters of function X") were excluded to ensure a fair comparison focused on semantic understanding.

BM25 (Keyword Search). BM25 is a classic information retrieval algorithm that ranks documents by term frequency and inverse document frequency. It serves as the traditional keyword-search baseline, representing systems without semantic understanding.

BGE Pure Vector Search. This baseline uses the same `BAAI/bge-small-zh-v1.5` embedding model as Vector-to-Graph but performs only vector similarity search without any knowledge graph fusion. By holding the embedding model constant, this baseline isolates the contribution of the graph fusion component.

Vector-to-Graph (Hybrid). The full hybrid system combining vector search, graph traversal, and weighted result fusion as described in Section 3.

All three systems were evaluated on the same 29 annotated queries using standard IR metrics: Precision@K, Recall@K, MRR, and NDCG@5. Table 5 shows the results.

Vector-to-Graph outperforms both baselines on all six metrics. Compared to BM25, the hybrid system achieves 2.4× higher Precision@5 and 2.8× higher Recall@10. Compared to the BGE pure vector search baseline—which uses the identical embedding model—the hybrid system achieves 1.9× higher Precision@5 and 2.1× higher Recall@10. The NDCG@5 improve-

Table 5: Baseline Comparison on 29 Concept-Level Queries

Metric	BM25	BGE Vector	Vector-to-Graph (Hybrid)
Precision@5	0.276	0.345	**0.655**
Precision@10	0.224	0.283	**0.600**
Recall@5	0.229	0.296	**0.586**
Recall@10	0.315	0.426	**0.875**
MRR	0.498	0.524	**0.609**
NDCG@5	0.367	0.408	**0.696**

ment from 0.408 (BGE) to 0.696 (Hybrid) represents a 70.6% relative gain attributable solely to graph fusion.

These results provide two key insights. First, embedding-based search (BGE) outperforms keyword-based search (BM25) by 25% in Precision@5, confirming the value of semantic understanding for concept-level queries. Second, and more importantly, adding knowledge graph fusion on top of the same embeddings yields a 90% improvement in Precision@5 (0.345 to 0.655), demonstrating that structural code relationships captured by the knowledge graph provide complementary information that significantly enhances retrieval quality.

6 Conclusion and Future Work

This paper presented Vector-to-Graph, a hybrid retrieval system for code documentation. The system combines vector search using BAAI/bge-small-zh-v1.5 embeddings with knowledge graph search built from Tree-sitter AST parsing. A keyword-based router classifies queries into four types and selects the appropriate retrieval strategy. Experiments on 121 test queries show 91.7% router accuracy, with error analysis revealing limitations of the keyword-based approach on ambiguous queries. The EnhancedTreeSitterParser adds 4 call edges (3.6ms overhead) to the base parser on the tested file, enabling function call relationship extraction.

Baseline comparison on 29 concept-level queries demonstrates that the hybrid approach achieves Precision@5 of 0.655 and NDCG@5 of 0.696, outperforming BM25 keyword search by 2.4× and pure vector search (using the identical BGE embedding model) by 1.9×. The 90% improvement from BGE-only to hybrid confirms that knowledge graph fusion provides substantial and quantifiable gains beyond semantic embedding alone.

Future work includes:

- Adding more programming language support (TypeScript, Go, Rust)
- Replacing the keyword-based router with a learned classifier (e.g., fine-tuned BERT) to improve robustness on ambiguous queries
- Implementing caching for frequently queried code elements
- Adding user feedback mechanisms for result relevance scoring
- Conducting end-to-end evaluation with a fully indexed real-world codebase and 100+ developer queries
- Extending the baseline comparison to include systems such as GraphRAG and HybridRAG on a standardized code retrieval benchmark

7 Acknowledgments

We thank our instructor for guidance and the open-source community for tools including Qdrant [8], Tree-sitter [7], NetworkX [3], and the Sentence-Transformers library [4].

References

[1] Anthropic. Model context protocol (mcp). 2024. Accessed: 2026-05-04.

[2] X. Chen and Y. Wang. Hybridrag: Integrating knowledge graphs and vector retrieval. 2024.

[3] NetworkX Developers. Networkx: Network analysis in python. 2024. Accessed: 2026-05-04.

[4] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. 2023.

[5] Microsoft Research. Graphrag: Graph-based retrieval augmentation generation. 2024.

[6] Safi Shamsi. Graphify: Ai coding assistant skill. 2026. Accessed: 2026-05-04.

[7] Tree sitter Contributors. Tree-sitter: A parser generator tool. 2024. Accessed: 2026-05-04.

[8] Qdrant Team. Qdrant: Vector similarity search engine. 2024. Accessed: 2026-05-04.

[9] Vincent Traag et al. From louvain to leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9(1), 2020.

A Complete Keyword Sets for Query Router

The router maintains the following keyword lists for query classification:

Table 6: Complete Vector Keywords

Keywords
是什么, 讲了什么, 总结, 介绍, 定义, 描述, 解释, 什么是, 怎么用, 如何使用

Table 7: Complete Graph Keywords

Keywords
代码, 函数, 类, 调用, 继承, 实现, 接口, 模块, 方法, 变量, 参数, 返回

Table 8: Complete Hybrid Keywords

Keywords
关系, 区别, 联系, 为什么, 怎么做到的, 有什么区别, 有什么关系, 为什么是, 如何实现

Table 9: Complete Direct Keywords

Keywords
你好, hello, hi, 谢谢, thanks